[spaCy Operations] (CheatSheet)

1. Basic Operations

- **Load a Language Model**: `nlp = spacy.load('en_core_web_sm')`
- **Create a Document Object**: `doc = nlp("This is a sentence.")`
- **Tokenization**: `[token.text for token in doc]`
- **Sentence Boundary Detection (SBD)**: `[sent.text for sent in doc.sents]`
- **Word Lemmatization**: `[token.lemma_ for token in doc]`
- **Word Shape Analysis**: `[token.shape_ for token in doc]`

2. Part-of-Speech (POS) Tagging

- **POS Tagging**: `[token.pos_ for token in doc]`
- **Detailed POS Tagging**: `[token.tag_ for token in doc]`
- **Visualizing POS**: `spacy.displacy.render(doc, style='dep', jupyter=True)`

3. Named Entity Recognition (NER)

- **Extract Named Entities**: `[(ent.text, ent.label_) for ent in doc.ents]`
- **Add Custom Named Entities**: `ruler = nlp.add_pipe('entity_ruler'); patterns = [{"label": "ORG", "pattern": "MyCompany"}]; ruler.add_patterns(patterns)`
- **Visualizing NER**: `spacy.displacy.render(doc, style='ent', jupyter=True)`

4. Dependency Parsing

- **Syntactic Dependency Parsing**: `[(token.text, token.dep_, token.head.text) for token in doc]`
- **Visualizing the Dependency Parse**: `spacy.displacy.render(doc, style='dep', jupyter=True)`

5. Word Vectors and Similarity

- **Word Embeddings (Vectors)**: `[token.vector for token in doc]`
- **Cosine Similarity Between Tokens**: `token1.similarity(token2)`
- **Checking if a Token has a Vector**: `token.has_vector`
- **Similarity Between Docs**: `doc1.similarity(doc2)`

By: Waleed Mousa

6. Customizing the Tokenizer

- **Adding Special Case Tokenization Rule**: `nlp.tokenizer.add_special_case("gimme", [{ORTH: "gim", LEMMA: "give"}, {ORTH: "me"}])`
- **Customizing Tokenizer Patterns**: `from spacy.tokenizer import Tokenizer; custom_nlp = spacy.blank("en"); custom_tokenizer = Tokenizer(custom_nlp.vocab)`

7. Rule-based Matching

- **Using the Matcher**: `from spacy.matcher import Matcher; matcher = Matcher(nlp.vocab); pattern = [{'LOWER': 'hello'}, {'LOWER': 'world'}]; matcher.add('HelloWorld', None, pattern)`
- **Using the PhraseMatcher for Large Terminology Lists**: `from spacy.matcher import PhraseMatcher; matcher = PhraseMatcher(nlp.vocab); matcher.add('OBAMA', None, nlp("Barack Obama"))`
- **Using the DependencyMatcher for Syntactic Relations**: `from spacy.matcher import DependencyMatcher; matcher = DependencyMatcher(nlp.vocab)`

8. Processing Pipelines

- **Viewing the Default Pipeline Components**: `nlp.pipe_names`
- **Adding a Component to the Pipeline**: `nlp.add_pipe(component)`
- **Creating a Custom Pipeline Component**: `def custom_component(doc): # do something; return doc`

9. Training and Updating Models

- **Updating the NER Model**: `other_pipes = [pipe for pipe in nlp.pipe_names if pipe != "ner"]; with nlp.disable_pipes(*other_pipes): optimizer = nlp.resume_training(); for itn in range(100): random.shuffle(TRAINING_DATA); for raw_text, entity_offsets in TRAINING_DATA: doc = nlp.make_doc(raw_text); example = Example.from_dict(doc, {"entities": entity_offsets}); nlp.update([example], drop=0.5, sgd=optimizer)`
- **Training a New Entity Type**: `TRAIN_DATA = [("Walmart is a leading e-commerce company", {"entities": [(0, 7, "ORG")]})]`

10. Extensions and Properties

By: Waleed Mousa

- **Setting Custom Metadata**: `Doc.set_extension('author', default=None); doc._.author = 'Some Author'`
- **Setting Custom Properties for Tokens**: `Token.set_extension('is_color', default=False); doc[0]._.is_color = True`

11. Language Detection

- **Language Detection using spaCy-langdetect**: `from spacy_langdetect import LanguageDetector; nlp.add_pipe(LanguageDetector(), name="language_detector", last=True)`
- **Getting Language Scores**: `doc._.language`

12. Text Classification

- **Text Categorizer Component**: `textcat = nlp.add_pipe("textcat")`
- **Adding Labels to Text Categorizer**: `textcat.add_label("POSITIVE"); textcat.add_label("NEGATIVE")`

13. Visualization Tools

- **Rendering with displaCy**: `spacy.displacy.serve(doc, style="dep")`
- **Customizing Visualizer**: `options = {"compact": True, "bg": "#09a3d5", "color": "#FFFFFF"}; spacy.displacy.render(doc, style="dep", options=options)`

14. Entity Linking

- **Knowledge Base Creation for Entity Linking**: `kb = KnowledgeBase(vocab=nlp.vocab, entity_vector_length=300)`
- **Adding Entities and Aliases to KB**: `kb.add_entity(entity_id, freq, entity_vector); kb.add_alias(alias, entities, probabilities)`

15. Lemmatization and Morphology

- **Custom Lemmatization**: `token.lemma_`
- **Accessing Morphological Features**: `token.morph`

16. Handling Large Volumes of Text

- **Processing Text in Batches**: `for doc in nlp.pipe(large_text_corpus, batch_size=500): process(doc)`

By: Waleed Mousa

- **Using nlp.pipe for Efficient Computation**: for doc in nlp.pipe(texts, disable=["tagger", "parser"]): analyze(doc)

17. Custom Tokenizer Rules

- **Prefix, Suffix, Infix Rules in Tokenizer**: nlp.tokenizer.prefix_search = custom_prefix_regex.search
- **Adding Special Case Rules**: nlp.tokenizer.add_special_case("don't", [{"ORTH": "do"}, {"ORTH": "n't", "LEMMA": "not"}])

18. Preprocessing before Tokenization

- **Custom Preprocessing Function**: def custom_preprocessor(text): # return preprocessed text

19. Custom Stop Words

- **Adding Custom Stop Words**: nlp.Defaults.stop_words |= {"my_custom_stopword"}
- **Removing Stop Words**: nlp.Defaults.stop_words -= {"my_custom_stopword"}

20. Numerical and Quantitative Features

- **Extracting Number of Entities**: len(doc.ents)
- **Extracting Number of Noun Chunks**: len(list(doc.noun_chunks))
- **Entity Frequency Distribution**: Counter([ent.label_ for ent in doc.ents])

21. Performance Optimization

- **Disabling Unnecessary Components**: nlp.disable_pipes('tagger', 'parser')
- **Using spaCy with GPU**: spacy.require_gpu()

22. Integration with Machine Learning Libraries

- **Converting spaCy Vectors to Scikit-learn/XGBoost Input**: X = [doc.vector for doc in docs]
- **Creating Pipelines with spaCy and Scikit-learn**: from sklearn.pipeline import Pipeline; text_clf = Pipeline([('spacy', SpacyVectorizer()), ('clf', LinearSVC())])

23. Custom NER Training

By: Waleed Mousa

- **Updating an Existing NER Model**: `ner = nlp.get_pipe("ner"); ner.add_label("NEW_LABEL"); optimizer = nlp.resume_training(); for itn in range(10): random.shuffle(TRAINING_DATA); losses = {}; for text, annotations in TRAINING_DATA: doc = nlp.make_doc(text); example = Example.from_dict(doc, annotations); nlp.update([example], drop=0.5, losses=losses, sgd=optimizer)`

24. Working with Multiple Languages

- **Loading Multilingual Models**: `nlp = spacy.load('xx_ent_wiki_sm')`
- **Language-specific Tokenization**: `nlp = spacy.load('de_core_news_sm')`

25. Saving and Loading Models

- **Saving a Model**: `nlp.to_disk('/path/to/model')`
- **Loading a Model from Disk**: `nlp_from_disk = spacy.load('/path/to/model')`

26. Entity Ruler for Adding Custom Rules

- **Creating an Entity Ruler**: `ruler = nlp.create_pipe("entity_ruler"); nlp.add_pipe(ruler)`
- **Adding Patterns to Entity Ruler**: `ruler.add_patterns([{"label": "ORG", "pattern": "Apple Inc."}])`

27. Matcher for Rule-based Matching

- **Defining Patterns for Matcher**: `pattern = [{"LOWER": "hello"}, {"IS_PUNCT": True}, {"LOWER": "world"}]`
- **Adding Patterns to Matcher**: `matcher.add("HelloWorld", None, pattern)`

28. Spans and SpanCategorizer

- **Creating Spans**: `span = doc[start:end]`
- **Training a SpanCategorizer**: `from spacy.training import Example; examples = [Example.from_dict(doc, annotations) for doc, annotations in data]; spancat = nlp.add_pipe("spancat"); spancat.add_label("ANIMAL")`

29. Customizing Visualizations

- **Customizing Displacy Style**: colors = {"ORG": "linear-gradient(90deg, #aa9cfc, #fc9ce7)"}; options = {"ents": ["ORG"], "colors": colors}; spacy.displacy.serve(doc, style="ent", options=options)

30. Advanced Text Processing

- **Sentence Segmentation**: about_doc = nlp(text); sentences = list(about_doc.sents)
- **Token-based Filtering**: [token for token in doc if not token.is_stop]
- **Noun Chunk Extraction**: [chunk.text for chunk in doc.noun_chunks]
- **Detecting Named Entities in Sentences**: [sent.ents for sent in doc.sents]

31. Performance and Scalability

- **Processing Large Volumes of Text Efficiently**: for doc in nlp.pipe(large_dataset, batch_size=50): process(doc)
- **Parallel Processing with spaCy**: from spacy.util import minibatch; batches = minibatch(large_dataset, size=10); for batch in batches: docs = nlp.pipe(batch)

32. Handling Complex Pipelines

- **Creating Complex Pipelines with Custom Components**: nlp.add_pipe(custom_component, after='ner')
- **Inspecting Pipeline Components**: nlp.analyze_pipes()

33. Custom Token Attributes

- **Setting Custom Attributes**: Token.set_extension('is_color', default=False); doc[0]._.is_color = True
- **Getting Custom Attributes**: token._.is_color

34. Contextual Spell Check

- **Correcting Spelling within Context**: contextual_spell_check.add_to_pipe(nlp)

35. Sentiment Analysis

- **Performing Sentiment Analysis**: doc._.polarity; doc._.subjectivity

By: Waleed Mousa

36. WordNet Integration

- **Accessing WordNet Synsets**: `from nltk.corpus import wordnet as wn; wn.synsets('dog')`

37. Custom Models and Fine-tuning

- **Fine-tuning Language Models**: `fine_tune_model(nlp, training_data)`

38. Advanced Entity Recognition

- **Fine-grained Entity Recognition**: `[(ent.text, ent.label_, ent.kb_id_) for ent in doc.ents]`

39. Training Language Models

- **Training Custom NER Models**: `train_ner_model(training_data)`
- **Custom Language Model Training**: `nlp.begin_training()`

40. Customization and Extension

- **Adding Custom Attributes to Docs, Spans, and Tokens**: `Token.set_extension('is_emoji', default=False)`
- **Writing Custom Pipeline Components**: `def custom_component(doc): # Modify doc here; return doc; nlp.add_pipe(custom_component)`

41. Processing and Transformation

- **Using spaCy with other NLP Libraries (e.g., Gensim, TextBlob)**: `TextBlob(doc.text).sentiment`
- **Custom Tokenizers**: `from spacy.tokenizer import Tokenizer; tokenizer = Tokenizer(nlp.vocab)`

42. Training Custom Text Categorizers

- **Adding a TextCategorizer to the Pipeline**: `textcat = nlp.create_pipe('textcat'); nlp.add_pipe(textcat)`
- **Training a Text Categorizer**: `train_text_categorizer(nlp, train_data)`

43. Word Similarity and Analogies

By: Waleed Mousa

- **Word Similarity**: token1.similarity(token2)
- **Finding Word Analogies**: most_similar(word, topn=3)

44. Integrating External Knowledge Bases

- **Entity Linking with Knowledge Bases**: nlp.add_pipe("entity_linker", config={"kb": {"@type": "knowledge_base", "name": "my_kb"}})

45. Advanced Data Structures

- **DocBin for Efficient Binary Serialization**: doc_bin = DocBin(docs=[doc1, doc2]); doc_bin.to_disk('/path/to/doc_bin.spacy')
- **Vocab, Lexemes, and StringStore**: nlp.vocab.strings['coffee']

46. Language Data and Tokenization

- **Accessing Language-specific Data**: nlp.vocab.morphology.tag_map
- **Custom Rules for Tokenization**: nlp.tokenizer.rules['don\'t'] = [{ORTH: "do"}, {ORTH: "n't", LEMMA: "not"}]

47. Advanced Visualization Techniques

- **Customizing Displacy Output**: options = {'compact': True, 'bg': '#09a3d5', 'color': 'white'}; spacy.displacy.render(doc, style='dep', options=options)
- **Visualizing Named Entities in Context**: spacy.displacy.serve(doc, style="ent")

48. Dealing with Non-Standard Language Data

- **Handling Emoji and Symbols**: token._.is_emoji; token._.emoji_desc
- **Working with Domain-Specific Language**: customize_tokenizer_for_domain(nlp)

49. Efficiency and Performance

- **Disabling Components for Speed**: nlp.disable_pipes('tagger', 'parser')
- **Efficiently Processing Large Volumes of Text**: for doc in nlp.pipe(large_corpus, batch_size=20): process(doc)

50. Integrating with Machine Learning Frameworks

By: Waleed Mousa

- **Converting spaCy features for Scikit-learn**: `X = np.array([doc.vector for doc in docs]); y = np.array([doc.cats for doc in docs]); clf.fit(X, y)`
- **Using spaCy Vectors for Deep Learning**: `embeddings = [token.vector for token in doc]; model.fit(embeddings, labels)`

By: Waleed Mousa